ASSIGNMENT 3: Software Development and Integration

Тастан Магжан | Astana IT University | 10.10.2026

В отчёте я рассматриваю требования задания на примере KazStyle - программы для определения стиля казахского текста. Названия и порядок десяти пунктов сохранены из оригинального задания [1].

1. Theoretical Part (Report/Presentation)

1.1. Stages and organization of the software development process. **Стр. 2.**

1.2. Criteria for selecting technologies and tools for software development. **Стр. 3.**

1.3. Overview and comparison of popular version control systems (e.g., Git and the GitHub platform). **Стр. 4.**

1.4. Key concepts and practices of continuous integration and continuous delivery (CI/CD). **Стр. 5.**

1.5. Features of scientific software development: requirements, environments, libraries, and best practices. **Стр. 6.**

2. Practical Part (Project/Repository)

2.1. Use of Git version control system; **Стр. 7.**

2.2. Hosting the project on GitHub (including README, license, issues, etc.); **Стр. 8.**

2.3. Setting up a basic CI/CD pipeline (e.g., using GitHub Actions, GitLab CI, Jenkins, etc.); **Стр. 9.**

2.4. Justification of chosen technologies (language, frameworks, libraries); **Стр. 10.**

2.5. Application of automated testing tools (if possible). **Стр. 11.**

Conclusion, challenges encountered and references. Стр. 12.

1.1. Stages and organization of the software development process.

Этапы и организация разработки

Разработка программы начинается с требований: нужно понять, что она должна делать и как проверить результат. Затем выбирается структура, пишется код, выполняются тесты и выпускается рабочая версия. После выпуска исправляются ошибки и добавляются функции. Эти этапы образуют жизненный цикл программы, или SDLC.

В KazStyle я использую итерационный подход. Сначала делаю рабочую часть, проверяю её на примерах и затем исправляю найденные проблемы. Для исследования это удобно, потому что заранее нельзя знать все ошибки корпуса и моделей.

Этап	Что сделано в KazStyle
Требования	Программа принимает казахский текст, определяет один из пяти стилей и позволяет сравнить модели.
Проектирование	Обработка данных, обучение, оценка, предсказание и сайт разделены на модули.
Реализация	Написаны очистка текста, подготовка выборок, обучение и обработка запросов сайта.
Тестирование	Проверяются функции очистки, сохранённые модели и сервер. Качество классификации оценивается отдельно.
Выпуск	Основной сайт использует фиксированный профиль моделей. Код хранится в GitHub.
Сопровождение	Ошибки и решения записываются в документации; новая обработка создаёт новую версию данных.

Например, расширенная модель получила высокий результат на внутреннем тесте, но ошиблась на школьном заявлении. Поэтому я не заменил ею основной профиль сайта. Для обновления важны и общая метрика, и поведение на конкретных примерах.

Состояние работы и принятые решения собраны в docs/PROJECT.md. Команды запуска находятся в docs/WORKFLOW.md. Это помогает продолжать работу без повторного изучения всех старых отчётов. Разработка идёт по итерациям, но я не называю её полноценным Scrum: формальных спринтов и командных ролей в проекте нет.

1.2. Criteria for selecting technologies and tools for software development.

Критерии выбора технологий и инструментов

Технологию нужно выбирать под задачу: учитывать доступные библиотеки, совместимость с существующим кодом, сложность доработки, переносимость и средства проверки. Для KazStyle особенно важны обработка казахского текста и работа с моделями машинного обучения.

В отчёте я сравнил Python, Java и JavaScript методом взвешенной суммы [7]. Каждому критерию назначается вес, а языку - оценка от 1 до 5. Один балл означает большие переделки, три - пригодность с доработками, пять - хорошее соответствие текущему проекту. Итог равен сумме произведений весов и оценок.

Критерий	Вес	Python	Java	JavaScript
Совместимость с ML-кодом	0,35	5	2	2
Простота доработки	0,20	4	3	4
Переносимость	0,20	5	4	4
Тестирование в проекте	0,15	5	3	3
Сохранение готового кода	0,10	5	3	3
Итог	1,00	4,80	2,85	3,05

Для Python расчёт такой: $0{,}35 \times 5 + 0{,}20 \times 4 + 0{,}20 \times 5 + 0{,}15 \times 5 + 0{,}10 \times 5 = 4{,}80$. Самый большой вес у совместимости с ML-кодом, потому что обработка данных и обучение уже реализованы на Python.

Для проверки устойчивости выбора я уменьшил этот вес до 0,20 и увеличил вес переносимости до 0,35. Python сохранил первое место: 4,80 против 3,15 у Java и 3,35 у JavaScript. То есть при такой смене приоритетов вывод не изменился.

Баллы и веса заданы мной для этой задачи. Это не измерение скорости языков. Некоторые критерии связаны между собой, поэтому таблица служит обоснованием выбора для готового проекта, а не доказательством универсального превосходства Python. Конкретные библиотеки и их роль приведены в пункте 2.4.

1.3. Overview and comparison of popular version control systems (e.g., Git and the GitHub platform).

Сравнение систем контроля версий

Система контроля версий сохраняет историю файлов. Она позволяет посмотреть изменения, найти версию до ошибки и вернуть нужное состояние. Для сравнения я взял Git и Subversion, или SVN. GitHub рассмотрен отдельно: это платформа размещения Git-репозиториев, а не другая система контроля версий [3].

Критерий	Git	SVN
Организация	Распределённая: обычный полный клон содержит историю.	Централизованная: основная история хранится на сервере.
Без доступа к серверу	Можно создавать локальные коммиты и изучать имеющуюся историю.	Можно редактировать рабочую копию; для коммита нужен сервер.
Передача изменений	Коммит сохраняется локально, затем отправляется через push.	Коммит отправляется в центральный репозиторий.
Ветвление	Локальные ветки удобны для отдельных изменений.	Ветки обычно организованы как каталоги в репозитории.
Для KazStyle	Уже используется и связан с GitHub Actions.	Не используется; переход не решает текущую задачу.

В моём проекте Git удобен тем, что я могу сохранить изменение и проверить его локально, а затем отправить в GitHub. Отдельная ветка позволяет работать над изменением, сохраняя основную версию. Перед объединением можно просмотреть разницу файлов и результаты тестов.

Роль GitHub

GitHub хранит удалённую копию репозитория и даёт общий адрес проекта. README объясняет запуск, Issues позволяют записывать задачи, pull request помогает обсуждать изменения, а Actions запускает проверки. Эти возможности дополняют Git. Наличие репозитория само по себе не означает, что все они уже используются.

Для KazStyle выбран Git с размещением на GitHub. Фактическая история коммитов описана в пункте 2.1, а состояние README, лицензии и Issues - в пункте 2.2. Такая схема позволяет отдельно оценить выбор инструмента и его применение.

1.4. Key concepts and practices of continuous integration and continuous delivery (CI/CD).

Основные понятия и практики CI/CD

CI означает непрерывную интеграцию. Разработчик отправляет изменения, после чего система автоматически получает код, готовит окружение и запускает проверки. Это помогает заметить ошибку сразу после изменения, а не перед демонстрацией программы.

Continuous delivery означает подготовку проверенного результата к выпуску. Решение о размещении может оставаться ручным. Continuous deployment означает автоматическое развёртывание после успешных проверок. Сохранение отчёта или архива ещё не означает, что сайт размещён на сервере.

Понятие	Простой смысл
Workflow	Сценарий автоматических действий, записанный в YAML.
Event	Событие запуска: например, push, pull request или ручная команда.
Job и runner	Задание и машина, на которой оно выполняется.
Step	Отдельный шаг: установка Python, запуск тестов или сохранение результата.
Artifact	Выходной файл запуска: например, журнал и HTML-отчёт.

Хорошая практика - запускать проверки в чистой среде, фиксировать версии зависимостей и сохранять журналы. Ошибка теста должна делать запуск неуспешным. Проверки стоит выполнять быстро, чтобы пользоваться ими после обычных изменений. Секреты и тяжёлые локальные файлы не нужно включать в репозиторий.

Для KazStyle выбран GitHub Actions: проект уже размещён на GitHub, поэтому не нужен отдельный сервер проверки. Сценарий находится рядом с кодом, а результат связан с конкретным коммитом [4].

В проекте уже реализован CI с подготовкой демонстрационного результата. Проверяются очистка, работа с путями и небольшой пример без весов моделей. Полное обучение в обычный запуск CI не включено: оно относится к отдельному исследовательскому эксперименту. Автоматическое развёртывание сайта пока не настроено. Реализация и ссылка на успешный запуск приведены в пункте 2.3.

1.5. Features of scientific software development: requirements, environments, libraries, and best practices.

Requirements: требования к научной программе

В KazStyle программа должна принять казахский текст, вернуть стиль и название модели, а при сравнении дать всем моделям один фрагмент. Очистка должна убрать ссылки и сохранить казахские буквы, имена и даты. Пустой ввод должен обрабатываться явно. Проверка идёт через примеры ввода, тесты функций и запросы сайта.

Environments: среда разработки и запуска

Код написан на Python в VS Code. Для сайта используется отдельное виртуальное окружение .venv, для GPU предусмотрено .venv-gpu. В CI создаётся чистая среда Python 3.12 на Ubuntu и Windows. Такое разделение позволяет проверять небольшой модуль без установки всего окружения обучения.

Libraries and dependency management: библиотеки и зависимости

Pandas используется для таблиц, scikit-learn - для TF-IDF и линейных моделей, PyTorch и Transformers - для нейросетей. Основные зависимости перечислены в requirements.txt. Для CI версии четырёх небольших зависимостей зафиксированы в configs/requirements-ci.txt. Инструкции GPU хранятся отдельно в docs/GPU.md.

Best practices: принятые правила работы

Воспроизводимость и учёт экспериментов. Сохраняются версия корпуса, параметры, seed, контрольные суммы, веса и предсказания. В сравнении использованы seed 42, 43 и 44. Каждый завершённый запуск хранится отдельно; старые результаты не заменяются новыми.

Проверка данных. Повторы и связанные документы группируются до разбиения на train, validation и test. Ссылки остаются в метаданных происхождения, а модель получает текст. TF-IDF обучается только на train. Иначе информация из оценки может попасть в обучение и завысить результат [6].

Тестирование и документация. Поведение функций проверяется через unittest. Изменения кода видны в Git, решения - в docs/PROJECT.md, команды - в docs/WORKFLOW.md. Проверки кода и качество классификации оцениваются отдельно. Предварительные метки корпуса не называются экспертной разметкой.

Ограничения остаются: полный сайт требует локальных весов и данных, а стиль текстов ещё нуждается в независимой человеческой проверке. В публичном репозитории доступен самостоятельный пример очистки; он проверяет код, но не заменяет исследовательский корпус.

2.1. Use of Git version control system;

Использование Git в практической работе

Для практической части выбран небольшой модуль обработки данных: kazstyle/demo.py. Он использует очиститель из KazStyle, принимает JSONL с полями id и text, убирает ссылки и HTML, исключает пустые записи и точные повторы. На выходе создаются очищенные данные, сводка и HTML-отчёт. Модуль можно запустить без моделей и GPU.

Код модуля, тесты, зависимости и описание включены в Git. Проверенная версия - коммит 2802e46. Он отправлен в ветку main публичного репозитория. К этому же коммиту относится успешный запуск GitHub Actions [2].

Действие	Для чего используется
git status --short	Показать изменённые и новые файлы перед сохранением.
git diff	Просмотреть конкретные изменения и проверить их состав.
git add	Выбрать файлы, которые должны войти в коммит.
git commit	Сохранить согласованный набор изменений с пояснением.
git push origin main	Отправить локальные коммиты в GitHub.
git log --oneline	Показать историю и идентификаторы версий.

В коммите 2802e46 добавлены переносимый пример, пять его тестов, сценарий CI и обновлённая инструкция запуска. Например, предыдущий коммит 6d68fd4 связан с очисткой текстов для следующей версии корпуса. По истории видно, что именно менялось, а не только конечное состояние файлов.

В .gitignore исключены виртуальные окружения, корпуса, веса и создаваемая папка build. Это уменьшает размер репозитория и отделяет исходный код от выходных файлов. Однако Git в таком виде не является резервной копией всех данных исследования: локальные корпуса и веса нужно сохранять отдельно.

Для этого изменения использован прямой push в main. Работа через отдельную ветку и pull request остаётся подходящим следующим шагом; выполненный code review или защиту ветки я здесь не заявляю.

2.2. Hosting the project on GitHub (including README, license, issues, etc.);

Размещение проекта и документация

Проект опубликован в открытом репозитории github.com/zzmaga/disser. В нём находятся исходный код, настройки, тесты, пример входных данных и описание запуска. Доступность репозитория и состояние следующих элементов проверены 10 октября 2026 года [2].

Элемент задания	Фактическое состояние
Публичный GitHub	Репозиторий доступен по общей ссылке.
README	Обновлён: назначение, запуск примера, зависимости сайта, модели, структура и показ CI.
License	Добавлена MIT-лицензия собственного кода и документации. У сторонних материалов свои условия.
Issues	Созданы три реальные задачи с описанием проблемы, действиями и критериями завершения.
Независимый запуск	Пример очистки работает без корпуса и весов. Для полного сайта нужны локальные ресурсы.

README содержит команды для Windows и Linux. После клонирования создаётся виртуальное окружение и устанавливаются зависимости из configs/requirements-ci.txt. Затем команда python -m kazstyle.demo --out-dir build/demo обрабатывает примеры. Результат можно посмотреть в build/demo/report.html.

Во входном примере пять записей. После очистки остаются три: одна запись исключается как повтор, другая содержит только ссылку. Пользователь получает cleaned.jsonl, summary.json и HTML-отчёт. Для повторного запуска нужен новый каталог результата, чтобы не перезаписать предыдущие файлы.

В README отдельно указано, что пример не определяет стиль. Полный классификатор использует сохранённые модели и корпус, которые не опубликованы в Git. Поэтому возможность запустить пример не выдаётся за возможность сразу развернуть весь сайт.

MIT-лицензия разрешает использовать и изменять собственный код проекта с сохранением уведомления об авторстве и текста лицензии. Она не меняет условия сторонних библиотек, текстов корпуса и предобученных моделей. Файл LICENSE и ссылки на задачи опубликованы коммитом 41a6c63.

Реальные задачи: #1 - ошибка на деловых жанрах, #2 - человеческая проверка меток, #3 - установка полного сайта. Все три открыты: я оформил оставшуюся работу, а не объявил её выполненной.

Прямая ссылка на описание: README с лицензией и задачами.

2.3. Setting up a basic CI/CD pipeline (e.g., using GitHub Actions, GitLab CI, Jenkins, etc.);

Реализованный pipeline в GitHub Actions

В проект добавлен workflow KazStyle CI. Он запускается при push и pull request в main, а также вручную. Один набор проверок выполняется на двух системах: Ubuntu и Windows, в обеих используется Python 3.12.

Шаг workflow	Что выполняется
Check out code	Загружается код проверяемого коммита.
Set up Python	Подготавливается Python 3.12.
Install demo dependencies	Устанавливаются фиксированные зависимости из requirements-ci.txt.
Run portable tests	Запускаются 14 тестов без локального корпуса и весов.
Run text cleaning demo	Обрабатываются пять записей и формируется результат.
Upload demo and test reports	Журналы и результат сохраняются как artifacts на 14 дней.

Подтверждённый результат: запуск № 38028767271 для коммита 2802e46 завершился успешно на обеих ОС. В каждом задании прошли 14 тестов без ошибок и пропусков. Проверены оба скачиваемых архива: в каждом есть журналы тестов, три очищенные записи, сводка и HTML-отчёт.

Конфигурация: .github/workflows/ci.yml.

Доказательство выполнения: успешный запуск GitHub Actions № 38028767271.

Сценарий проверки возвращает ошибку, если тесты упали, были пропущены или найдено меньше 14 тестов. Поэтому пустой либо неполный запуск не считается успешным. Версии используемых Actions закреплены идентификаторами коммитов.

На демонстрации я открываю этот запуск, показываю шаг Run portable tests и его итог. Затем скачиваю artifact kazstyle-demo-ubuntu-latest или вариант Windows, распаковываю его и открываю demo/report.html.

Реализованы автоматическая проверка и подготовка демонстрационного результата. Публикация сайта и развёртывание новых весов не настроены. Поэтому я показываю работающий CI, а не заявляю готовый процесс автоматического развёртывания всего KazStyle.

2.4. Justification of chosen technologies (language, frameworks, libraries);

Обоснование языка, фреймворков и библиотек

Я выбрал Python по сравнению из пункта 1.2: он подходит существующему коду и позволяет использовать уже выбранные библиотеки обработки текста и машинного обучения. Практический модуль использует тот же очиститель, что основной проект, поэтому проверяется полезная часть программы, а не отдельный несвязанный пример.

Технология	Зачем выбрана в KazStyle
Python	Один язык для обработки данных, моделей, сервера и тестов.
Beautiful Soup	Разбирает HTML при очистке текста. Используется основным очистителем и примером.
Lingua	Помогает проверять язык текста; используется в проверках качества данных.
unittest	Стандартный инструмент Python для автоматических проверок.
Pandas и scikit-learn	Таблицы корпуса, TF-IDF и линейные классификаторы полного проекта.
PyTorch и Transformers	Дообучение и использование нейросетевых моделей полного проекта.
HTML, CSS, JavaScript	Простой интерфейс локального сайта без отдельного frontend-фреймворка.
GitHub Actions	Запуск проверок рядом с Git-репозиторием, без собственного CI-сервера.

Для маленького модуля не нужны PyTorch, GPU или веб-фреймворк. Он запускается из командной строки, читает JSONL и создаёт файлы. Отдельный список зависимостей CI уменьшает установку и позволяет проверить пример на чистой машине.

В полном проекте обязанности разделены: kazstyle/data отвечает за данные, training и models - за обучение, inference - за предсказание, api и frontend - за сайт. Поэтому можно исправить очистку, не переписывая интерфейс, и проверить новые веса отдельным профилем.

Преимущество такого выбора - совместимость с готовым кодом и возможность быстро проверить небольшую часть. Ограничение - разные требования к окружению: лёгкий пример запускается самостоятельно, а полный сайт требует дополнительных библиотек и сохранённых моделей. Установка зависимостей сама по себе не создаёт эти модели.

2.5. Application of automated testing tools (if possible).

Применение автоматизированного тестирования

В проекте используется unittest, стандартный инструмент Python. Тест вызывает функцию и сравнивает ответ с ожидаемым результатом. Для программы очистки можно заранее задать текст со ссылкой и проверить, что ссылка исчезла, а казахские буквы сохранились [5].

Набор тестов	Количество	Что проверяется
test_quality.py	6	Ссылки, HTML, контакты, язык, подписи и сохранность имён, дат и букв.
test_storage.py	3	Чтение активных и архивных путей, а также запрет выхода за допустимый каталог.
test_demo.py	5	Формат записей, повторы, результат CLI и защита от перезаписи выходного каталога.
Всего в CI	14	На Windows и Ubuntu все прошли без ошибок и пропусков.

В тестах модуля есть правильный и неправильный ввод. Проверяются повторяющиеся ID, некорректный JSON и попытка записать результат в уже существующую папку. Отдельный тест заменяет только URL и подтверждает, что очищенный текст не изменился. Проверяется и настоящий запуск команды, а не только вызов функции.

Локально этот набор прошёл в отдельной копии исходников без моделей, корпуса и закрытых отчётов. Затем тот же сценарий выполнился в GitHub Actions. Команда запуска - python scripts/ci.py. Результат сохранён в ci-tests.txt и ci-tests.json внутри архивов запуска из пункта 2.3.

В проекте есть и более широкий локальный набор, который проверяет модели и HTTP. В сохранённом журнале от 6 октября прошли 76 тестов. Это другой запуск: число 76 не относится к текущему CI и не является процентом покрытия кода.

Тесты подтверждают поведение программы, но не правильность определения стиля любого текста. Для этого отдельно сравниваются предсказания с метками корпуса. В исследовании используются общие входы, отдельная validation-часть и несколько seed. Подробные метрики моделей не вынесены в отдельную главу, потому что этот пункт задания посвящён инструментам автоматического тестирования.

Ограничение текущего CI: он не загружает тяжёлые веса и не проверяет качество классификации. Такие проверки выполняются локально по исследовательскому плану. Успешный CI не заменяет оценку модели и человеческую проверку разметки.

Conclusion, challenges encountered and references

Краткий вывод

Для задания я использовал KazStyle и выделил из него самостоятельный модуль очистки. Код опубликован в GitHub, запуск описан в README, настроены проверки на Windows и Ubuntu. В обоих заданиях CI прошли 14 тестов; выходные файлы доступны для просмотра. Я обосновал выбор технологий и связал этапы разработки с конкретной работой в проекте.

Challenges encountered: возникшие трудности

Главная трудность - полный проект зависит от локальных весов и корпуса. Для проверки на чистой машине был выделен небольшой пример с отдельными зависимостями. Ещё одна проблема - ссылки, HTML и повторы в текстах. Их обработка проверяется тестами. Высокая метрика модели при этом не исключает ошибок на отдельных жанрах.

В публичном репозитории теперь есть README, MIT-лицензия, три реальные задачи в Issues и работающий CI. Полный сайт требует локальных весов, автоматическое развёртывание не настроено. Независимая проверка меток и ошибки отдельных жанров остаются исследовательскими задачами. Их оформление в Issues не означает завершение самой диссертации.

Источники и ссылки

[1] ASSIGNMENT 3. Software Development and Integration. Предоставленное задание, 1 страница.

[2] KazStyle: репозиторий, MIT License, Issues, конфигурация CI, успешный запуск. Код CI: 2802e46; лицензия и README: 41a6c63.

[3] Chacon S., Straub B. Pro Git. About Version Control.

[4] GitHub Docs. Understanding GitHub Actions.

[5] Python Documentation. unittest: Unit testing framework.

[6] scikit-learn. Common pitfalls and recommended practices.

[7] Department for Communities and Local Government. Multi-criteria analysis: a manual. 2009, раздел 4.3.3, с. 22.

Состояние репозитория и CI проверено 10.10.2026. Упоминание 76 локальных тестов относится к журналу от 06.10.2026. Все ссылки в PDF открываются нажатием.